Quiz P7 Pemrograman Berorientasi Objek

Nama : Torikh Abdullah Naser
NIM : 2410512031
Kelas : A-S1 Sistem Informasi

Jawaban Analisis Tugas Pertemuan 7 (Studi Kasus)
Soal 1: Sistem Perpustakaan

A. Analisis

1. Apa yang dimaksud dengan class dan object dalam konteks kasus ini?
   Class (Kelas): Cetak biru (blueprint) atau kerangka untuk membuat sebuah buku. Class Buku mendefinisikan sifat-sifat umum (atribut seperti judul, pengarang, status ketersediaan) dan perilaku nyata (method seperti pinjam, kembalikan, tampil info) yang akan dimiliki oleh semua buku di dalam sistem.
   Object (Objek): Merupakan wujud (instance) hasil nyata dari class Buku. Jika class adalah rancangannya, maka object adalah wujud buku fisiknya. Sebuah object buku memiliki nilai atribut atau data yang berbeda-beda dan spesifik antar satu dan lainnya.

2. Berikan contoh object nyata dari class yang akan dibuat
   Object 1: Sebuah buku spesifik dengan judul "Laskar Pelangi", pengarang "Andrea Hirata", dan ketersediaannya true (tersedia).
   Object 2: Sebuah buku spesifik dengan judul "Bumi Manusia", pengarang "Pramoedya Ananta Toer", dan ketersediaannya false (sedang dipinjam).

3. Atribut apa saja yang diperlukan? Judul buku (tipe datanya String), Pengarang (String), dan Status Ketersediaan (boolean).

4. Method apa saja yang dibutuhkan? pinjamBuku(), kembalikanBuku(), dan tampilInfo().

5. Jika semua logika ditulis di main() tanpa class, apa kelemahannya? Kode programnya menjadi sangat padat di satu tempat (spaghetti code), variabel-variabel yang merepresentasikan bermacam-macam buku menjadi gampang tertukar karena tidak ada pengelompokan yang jelas, sangat banyak redundansi/pengulangan kode, dan akan sangat sulit jika dimodifikasi karena sistemnya tidak rapi.

B. Implementasi:

Buku.java

```java
package quiz_p7;

public class Buku {
    private String judul;
    private String pengarang;
    private boolean tersedia;

    public Buku(String judul, String pengarang) {
        this.judul = judul;
        this.pengarang = pengarang;
        this.tersedia = true;
    }
  Ctrl+L for Agent
    public void pinjamBuku() {
        if (tersedia) {
            tersedia = false;
            System.out.println("Buku '" + judul + "' berhasil dipinjam.");
        } else {
            System.out.println("Maaf, buku '" + judul + "' sedang tidak tersedia.");
        }
    }

    public void kembalikanBuku() {
        if (!tersedia) {
            tersedia = true;
            System.out.println("Buku '" + judul + "' berhasil dikembalikan.");
        } else {
            System.out.println("Buku '" + judul + "' sudah tersedia (belum dipinjam).");
        }
    }

    public void tampilInfo() {
        System.out.println("--- Informasi Buku ---");
        System.out.println("Judul: " + judul);
        System.out.println("Pengarang: " + pengarang);
        System.out.println("Status: " + (tersedia ? "Tersedia" : "Dipinjam"));
        System.out.println("----------------------");
    }
}
```

MainSoal1.java

```java
package quiz_p7;

public class MainSoal1 {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("=== Simulasi Sistem Perpustakaan ===");

        // Membuat 2 object Buku
        Buku buku1 = new Buku("Laskar Pelangi", "Andrea Hirata");
        Buku buku2 = new Buku("Bumi Manusia", "Pramoedya Ananta Toer");

        // Menampilkan info awal
        buku1.tampilInfo();
        buku2.tampilInfo();

        // Simulasi peminjaman dan pengembalian
        System.out.println("\n>> Meminjam buku1:");
        buku1.pinjamBuku();

        System.out.println("\n>> Mencoba meminjam buku1 lagi:");
        buku1.pinjamBuku();

        System.out.println("\n>> Menampilkan info setelah buku1 dipinjam:");
        buku1.tampilInfo();

        System.out.println("\n>> Mengembalikan buku1:");
        buku1.kembalikanBuku();

        System.out.println("\n>> Menampilkan info setelah buku1 dikembalikan:");
        buku1.tampilInfo();
    }
}
```

Output kode

```
PS D:\kuliah\smt 4\prak pbo>  & 'C:\Program Files\Eclipse Adoptium\
orage\742f7bab6972091c9de5bfda8d2d49b3\redhat.java\jdt_ws\prak pbo_
=== Simulasi Sistem Perpustakaan ===
--- Informasi Buku ---
Judul: Laskar Pelangi
Pengarang: Andrea Hirata
Status: Tersedia
---------------------
--- Informasi Buku ---
Judul: Bumi Manusia
Pengarang: Pramoedya Ananta Toer
Status: Tersedia
---------------------

>> Meminjam buku1:
Buku 'Laskar Pelangi' berhasil dipinjam.

>> Mencoba meminjam buku1 lagi:
Maaf, buku 'Laskar Pelangi' sedang tidak tersedia.

>> Menampilkan info setelah buku1 dipinjam:
--- Informasi Buku ---
Judul: Laskar Pelangi
Pengarang: Andrea Hirata
Status: Dipinjam
---------------------

>> Mengembalikan buku1:
Buku 'Laskar Pelangi' berhasil dikembalikan.

>> Menampilkan info setelah buku1 dikembalikan:
--- Informasi Buku ---
Judul: Laskar Pelangi
Pengarang: Andrea Hirata
Status: Tersedia
---------------------
PS D:\kuliah\smt 4\prak pbo>
```

C. Analisis Lanjutan

Jika terdapat 100 buku:

1. Mengapa penggunaan object lebih baik dibandingkan variabel biasa?
   Jika menggunakan pendekatan variabel biasa (prosedural), kita harus membuat hingga ratusan pasang variabel seperti judul1, judul2... hingga judul100. Pendekatan ini sangat repot dan membingungkan untuk dikelola. Dengan perancangan object (PBO / OOP), kita hanya perlu membuat class Buku satu kali saja, lalu menciptakan 100 object dari rancangan yang sama (bisa dimasukkan ke dalam list/array Buku[]). Semua fungsi peminjaman dan pengembalian akan terkait langsung pada setiap objek tersebut sehingga struktur lebih rapi.

2. Jelaskan dari sisi skalabilitas dan maintainability:

- Skalabilitas (Kemampuan Berkembang): Jika jumlah buku bertambah ribuan atau jika ingin menambah informasi "penerbit" dan "tahun terbit", kita cukup memodifikasi di dalam cetak biru class Buku. Dengan demikian, menambah fitur tidak akan merusak keseluruhan logika awal.
- Maintainability (Perawatan Sistem): Jika suatu hari ada kesalahan logika proses (bug) pada pencatatan pinjam buku, kita cukup memeriksa dan memperbaiki isi method pinjamBuku() di dalam class Buku. Satu perbaikan di tempat ini akan otomatis berlaku pada semua objek Buku di dalam sistem.

Soal 2: Pertarungan Game

A. Analisis Konsep

1. Mengapa method attack() sebaiknya berada di dalam class Character, bukan di main()? Karena tindakan menyerang (attack) merupakan tanggung jawab alami (perilaku) dari objek sebuah entitas karakter game. Dalam paradigma OOP dikenal prinsip Encapsulation, yaitu menyatukan atribut sebuah benda dan sifat operasionalnya ke dalam satu kesatuan Class. Jika hal tersebut ditulis di main(), fungsi main() harus memanipulasi variabel setiap karakter secara primitif, menyalahi aturan pemisahan fungsi dan konsep encapsulation.

2. Jelaskan hubungan antara:
   a. object dengan method: Method adalah kapabilitas/aksi milik sebuah object. Pemanggilan method berguna untuk mengubah state/status sebuah object atau merespons suatu perintah berdasarkan nilainya masing-masing.
   b. object dengan object lain (dalam kasus serangan): OOP dikenal dengan model Message Passing atau interaksi objek. Pada kasus ini, object Character pertama akan memanggil method attack miliknya, lalu di dalam proses serangan tersebut, method akan "berkomunikasi" mengubah status HP (darah) dari object Character target kedua dari luar melalui referensi parameter.

3. Apa yang terjadi jika parameter attack() bukan object, tetapi hanya integer? Jika yang diterima hanya berupa integer (contoh: ID musuh atau angka attack langsung), method menjadi tidak tahu karakter mana secara struktural yang sedang ia serang sehingga tidak bisa terhubung langsung mengubah HP lawannya. Karena parameter yang dilempar berupa object class itu sendiri (Character target), barulah method memiliki kapabilitas utuh memerintahkan target untuk menerima damage (target.takeDamage(...)).

B. Implementasi

Character.java

```java
package quiz_p7;

public class Character {
    private String nama;
    private int hp;
    private int attackPower;

    public Character(String nama, int hp, int attackPower) {
        this.nama = nama;
        // Pencegahan HP negatif saat inisialisasi karakter
        this.hp = Math.max(hp, 0);
        this.attackPower = attackPower;
    }

    // Method privat untuk menangani pengurangan HP dari serangan karakter lain
    private void takeDamage(int damage) {
        this.hp -= damage;
        if (this.hp < 0) {
            this.hp = 0; // Solusi untuk mencegah HP menjadi negatif (Soal 2 Bagian C point 1b)
        }
    }

    public void attack(Character target) {
        if (this.hp <= 0) {
            System.out.println(this.nama + " tidak bisa menyerang karena sudah kalah.");
            return;
        }

        System.out.println(this.nama + " menyerang " + target.getNama() + " dengan kekuatan " + attackPower + "!");
        target.takeDamage(this.attackPower);

        if (target.getHp() == 0) {
            System.out.println(target.getNama() + " telah dikalahkan!");
        }
    }

    public void displayStatus() {
        System.out.println("Status " + nama + ": HP = " + hp + ", Attack Power = " + attackPower);
    }

    // Getter methods
    public String getNama() {
        return nama;
    }

    public int getHp() {
        return hp;
    }
}
```

MainSoal2.java

```java
package quiz_p7;

public class MainSoal2 {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("=== Simulasi Pertarungan ===");

        // Membuat minimal 2 object Character
        Character hero = new Character("Arthur", 100, 25);
        Character monster = new Character("Goblin", 60, 15);

        // Menampilkan status awal
        System.out.println(">> Status Awal:");
        hero.displayStatus();
        monster.displayStatus();

        // Simulasi interaksi antar object
        System.out.println("\n>> Pertarungan Dimulai!");
        hero.attack(monster);
        monster.displayStatus();

        System.out.println();
        monster.attack(hero);
        hero.displayStatus();

        System.out.println();
        hero.attack(monster);
        monster.displayStatus();

        System.out.println();
        // Serangan penentu (HP Goblin akan dibuat tidak negatif sesuai penerapan design)
        hero.attack(monster);
        monster.displayStatus();
    }
}
```

Output Kode

```
PS D:\kuliah\smt 4\prak pbo>  & 'C:\Program File
orage\742f7bab6972091c9de5bfda8d2d49b3\redhat.ja
=== Simulasi Pertarungan ===
>> Status Awal:
Status Arthur: HP = 100, Attack Power = 25
Status Goblin: HP = 60, Attack Power = 15

>> Pertarungan Dimulai!
Arthur menyerang Goblin dengan kekuatan 25!
Status Goblin: HP = 35, Attack Power = 15

Goblin menyerang Arthur dengan kekuatan 15!
Status Arthur: HP = 85, Attack Power = 25

Arthur menyerang Goblin dengan kekuatan 25!
Status Goblin: HP = 10, Attack Power = 15

Arthur menyerang Goblin dengan kekuatan 25!
Goblin telah dikalahkan!
Status Goblin: HP = 0, Attack Power = 15
PS D:\kuliah\smt 4\prak pbo>
```

C. Analisis Lanjutan

1. Jika HP bisa menjadi negatif:
    a. Apakah ini masalah desain? Ya, secara fungsional ini menandakan adanya permasalahan desain (Bug Logical). HP game tidak ada yang di bawah 0 (jika mati berarti 0). Hal ini mengidentifikasikan bahwa perancang lupa membuat Guard logic / Filter Validasi kondisional sebelum variabel berkurang dari hasil pengurangan damage.
    b. Bagaimana cara memperbaikinya? Dengan menyertakan validasi langsung di dalam class Character (misalnya pada bagian atau method untuk menerima damage). Contoh dalam kode Java di class:

    ```
    if (this.hp < 0) {
    this.hp = 0;
    }
    ```

2. Jika game berkembang:
    a. Mengapa desain berbasis object lebih mudah dikembangkan? Jika game terus ditingkatkan perilisannya (ada penambahan Boss Monster, job karakter seperti kesatria dan pemanah, sistem item dll), desain PBO memungkinkan hal ini diselesaikan melalui prinsip seperti Inheritance (Pewarisan sifat). Misalnya Class Mage adalah turunan Character tetapi memiliki jurus magic, dan Polymorphism (kemampuan benda merespons dengan metode uniknya). Hal seperti ini mencegah pengulangan ribuan baris kode, membuat sistem lebih adaptif tanpa harus mengedit banyak kode sumber dasar yang telah dikerjakan di awal.

Link Source code: https://github.com/Torikh42/pbo